UNIVERSIDAD TECNOLÓGICA DEL PERÚ

GUÍA PRÁCTICA DE LABORATORIO – TEORÍA EN COMPUTACIÓN

Curso: Teoría en Computación

Docente: Francisco Cabello

Semana: 01

Tema: Introducción a la Teoría de la Computación
Integrantes:

- Félix Francisco Córdova Pachas U23234449
- Solange Ariana Cepeda Laynes U23231714
- Alonso Zegarra Velasquez U23210102

1. OBJETIVO

Comprender el funcionamiento de los traductores y el sistema computacional mediante actividades prácticas guiadas.

2. ACTIVIDAD 1: TEORÍA DE LA COMPUTACIÓN

TAREAS DEL ESTUDIANTE:

1. Lea el material de la semana 1.

2. Defina con sus propias palabras qué es la teoría de la computación.

3. Indique su finalidad.

4. Explique su relación con los lenguajes de programación.

SOLUCIÓN PASO A PASO

Es la base teórica que permite entender como una computadora interpreta y ejecuta un programa mediante los distintos lenguajes de programación. Además, define como se construyen y analizan dichos lenguajes de programación, asegurando que tengan reglas claras y consistentes.

3. ACTIVIDAD 2: SISTEMA COMPUTACIONAL

TAREAS DEL ESTUDIANTE:

1. Identifique hardware, software y usuario.

2. Describa cada componente.

3. Explique cómo interactúan.

SOLUCIÓN PASO A PASO

El hardware es todo lo físico (El procesador, la ram, la fuente de poder, el case, etc)

El software es lo que hace funcionar en si la misma PC (El Sistema operativo, los programas, los drivers)

El usuario es la persona que este usando la computadora y los programas a la vez

Su interacción es netamente a través de los dispositivos de entrada, como el teclado, el mouse, el monitor, junto con los drivers, como el driver de sonido, el driver del mouse, etc

4. ACTIVIDAD 3: TRADUCTORES

TAREAS DEL ESTUDIANTE:

1. Defina qué es un traductor.

2. Identifique tipos: compilador, intérprete, ensamblador.

3. Dé un ejemplo de cada uno.

SOLUCIÓN PASO A PASO

Es un programa que permite convertir de código fuente de alto nivel a lenguaje objeto como código de maquina o intermedio

- Compilador: Programa que traduce todo el programa a la vez y genera un archivo ejecutable como .exe
- Interprete: Programa que traduce el programa línea por línea, no genera un archivo ejecutable
- Ensamblador: Trabaja con lenguajes de bajo nivel y se ejecuta por el procesador

Compilador: GCC compila programas en C y C++

Interprete: Node.js ejecuta JavaScript fuera del navegador

Ensamblador: MASM ensamblador de Microsoft

5. ACTIVIDAD 4: PROCESO DE EJECUCIÓN

TAREAS DEL ESTUDIANTE:

1. Analice el código print('Hola').

2. Explique cómo se traduce.

3. Dibuje el flujo de ejecución.

SOLUCIÓN PASO A PASO

El interprete de Python en este caso, actúa como traductor: convierte el código de alto nivel en instrucciones de bajo nivel para que la maquina lo entienda.

El flujo sería el siguiente:

Primero escribo la línea de código en lenguaje Python en One Compiler, luego le doy al botón Run y en output se muestra el resultado

Código Fuente > Traductor > Resultado

6. ACTIVIDAD 5: COMPARACIÓN

TAREAS DEL ESTUDIANTE:

1. Compare compilador e intérprete.

2. Indique ventajas y desventajas.

3. Justifique cuál es mejor en cada caso.

SOLUCIÓN PASO A PASO

El compilador genera un .exe, mientras que el intérprete traduce la línea una por una, mientras se ejecuta el programa

El compilador tiene las siguientes ventajas: Ayuda al rendimiento y a la distribución profesional, es rápida al ejecutar, ayuda a proteger el código fuente, y el software puede hacer mejoras de manera automática para ocupar menos memoria.

El intérprete es más portable, quiere decir que no importa el sistema operativo siempre y cuando se tenga el mismo intérprete. El tamaño también es reducido, tiene debug (depuración) para ayudarte a identificar los errores.

Si son complejos o con gráficos 3D: Compilador, Necesitamos la velocidad y las mejoras automáticas

Si son scripts automatizados: Intérprete, Escribimos códigos rápidos y ver resultados inmediatos

Si debe ser una aplicación muy segura: Compilador, Es mucho más difícil de hacer ingeniería inversa

Si es una herramienta que debe ser multiplataforma: Intérprete, Solo necesitas el mismo intérprete, no importa si son distintos sistemas operativos.

7. ACTIVIDAD 6: PRÁCTICA EN PYTHON

TAREAS DEL ESTUDIANTE:

1. Ejecute print('Hola mundo').

2. Identifique el tipo de traductor.

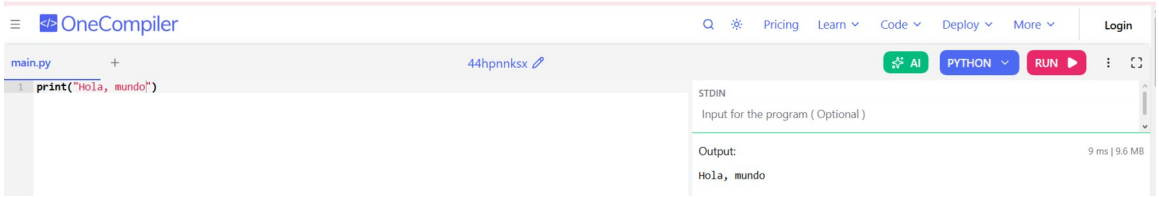
3. Explique el proceso.

SOLUCIÓN PASO A PASO

Al ejecutar sale: Hola mundo

Es un intérprete ya que lo lee línea por línea

Escribir el código, el intérprete lo traduce, traducción a un lenguaje intermedio, ejecución inmediata y salida en pantalla



8. ACTIVIDAD 7: REFLEXIÓN

TAREAS DEL ESTUDIANTE:

1. Explique importancia de la teoría de la computación.

2. Indique aplicaciones reales.

3. Explique rol de traductores.

SOLUCIÓN PASO A PASO

Nos permite entender como una computadora interpreta y ejecuta un programa escrito en un lenguaje de programación.

Compiladores (Python o Java), app como Google Maps o Waze los programas de IA.

Convierten el lenguaje de alto nivel en instrucciones de bajo nivel permitiendo que la computadora pueda ejecutarlas.

9. ACTIVIDAD 8: ANÁLISIS COMPARATIVO AVANZADO

TAREAS DEL ESTUDIANTE:

1. Compare compilador vs intérprete a nivel de rendimiento, memoria y detección de errores.

2. Analice en qué escenarios reales usaría cada uno.

3. Justifique técnicamente su elección.

SOLUCIÓN PASO A PASO

Comparativa de Nivel Técnico

A continuación, se desglosan los tres pilares solicitados para entender cómo impactan en el sistema.

Criterio	Compilador	Intérprete
Rendimiento	**Alto.** El código se traduce a lenguaje máquina antes de usarse, permitiendo que la CPU lo ejecute a máxima velocidad.	**Moderado/Bajo.** El software traduce el código línea por línea mientras corre, lo que genera un "retraso" constante.
Uso de Memoria	**Eficiente en ejecución.** Aunque el proceso de compilación consume recursos, el programa final es ligero y optimizado.	**Exigente.** El intérprete debe estar siempre en RAM junto al código fuente y las tablas de datos para funcionar.
Detección de Errores	**Preventiva (Estática).** Los errores se encuentran antes de que el programa exista. Si algo falla, el ejecutable no se crea.	**En marcha (Dinámica).** Los errores aparecen mientras el usuario usa el programa. Puede fallar de repente tras funcionar bien un rato.

2. Análisis de Escenarios Reales

Escenario A: Desarrollo de un Motor de Videojuegos (AAA)

- **Elección:** Compilador (Ej. C++, Rust).

- **Razón:** En un videojuego, el renderizado de fotogramas ocurre en milisegundos. No puedes permitirte el retraso de un intérprete traduciendo código en medio de una batalla épica. Necesitas acceso directo al hardware (GPU/CPU).

Escenario B: Análisis de Datos y Machine Learning

- **Elección:** Intérprete (Ej. Python).
- **Razón:** Los científicos de datos necesitan probar hipótesis rápidamente. Un intérprete permite ejecutar una celda de código, ver el gráfico resultante y ajustar la variable inmediatamente sin esperar a que todo el proyecto se recompile.

Escenario C: Scripts de Automatización de Servidores

- **Elección:** Intérprete (Ej. Bash, Python).
- **Razón:** La portabilidad es clave. Un script debe poder ejecutarse en diferentes servidores sin necesidad de compilar un binario específico para cada arquitectura de procesador.

---

3. Justificación Técnica de la Elección

La decisión entre compilar o interpretar depende de la **prioridad del proyecto**:

1. **Optimización de Recursos:** En sistemas con hardware limitado (como un microcontrolador o un dispositivo médico), la **compilación** es vital para exprimir al máximo la memoria y la CPU disponibles.

2. **Velocidad de Desarrollo vs. Velocidad de Ejecución:** Si el tiempo de entrega es crítico y el código cambiará constantemente, un **intérprete** acelera el trabajo del programador. Si la velocidad de respuesta del programa final es lo que importa (ej. sistemas de frenado ABS), la **compilación** no es negociable.

3. **Seguridad y Control:** Los lenguajes compilados suelen dar un control manual sobre la memoria, mientras que los interpretados ofrecen capas de seguridad que evitan que el programador cometa errores graves de gestión de hardware, a costa de un poco de velocidad.

10. ACTIVIDAD 9: DISEÑO DE UN PROCESO DE TRADUCCIÓN

TAREAS DEL ESTUDIANTE:

1. Diseñe un flujo completo desde código fuente hasta ejecución.

2. Identifique cada etapa del proceso.

3. Explique qué ocurre en cada etapa.

SOLUCIÓN PASO A PASO

Código fuente–> Preprocesador -> Compilación –> Ensamblado –> Enlazado -> Carga y Ejecución

- Código fuente: Es el archivo de texto escrito por el desarrollador en un lenguaje de alto nivel. Contiene las instrucciones lógicas, pero la computadora no puede entenderlo directamente
- Preprocesador: Interpreta directivas especiales
- Compilación: Analiza el código preprocesado en busca de errores de sintaxis y lógica
- Ensamblado: Toma el archivo anterior y lo traduce a código binario puro
- Enlazado: Combina el código y da como resultado el archivo ejecutable final
- Carga y Ejecución: El procesador ejecuta las instrucciones una a una

11. ACTIVIDAD 10: CASO PRÁCTICO AVANZADO

TAREAS DEL ESTUDIANTE:

1. Analice el siguiente código: print(5+3)

2. Explique cómo lo procesa un intérprete.

3. Explique cómo lo procesaría un compilador.

4. Compare ambos procesos.

SOLUCIÓN PASO A PASO

Esa línea de código muestra el resultado de esa suma, que sería 8.

El intérprete Python lee línea por línea y traduce la instrucción en lenguaje máquina.

El compilador traduce todo el programa de una sola vez.

El intérprete va fase por fase, mientras que el compilador lo traduce completo y genera un ejecutable.